

Finding the Global Minimum of the 2D Ackley Function

with Genetic Algorithm

Course Instructor

Dr. Al Mahmud

Professor, Department of CSE, KUET

Presented by

2107067

2107078

2107091

2107100

2107108

2107115

A Genetic Algorithm that hunts for a single point in a 2D landscape

01

Goal

Find the (x1, x2) point that drives the 2D Ackley function as close as possible to its known global minimum $f(0,0) = 0$.

02

Approach

Evolve a population of 50 candidate points over up to 100 generations using selection, crossover, mutation and elitism — no gradients required.

03

Benchmark

Compare the GA's global search against SciPy's Nelder-Mead, a classical local, derivative-free optimizer, from a fixed starting point.

04

Deliverable

A tested Python codebase (NumPy/Matplotlib/SciPy/pytest) that prints progress, saves plots, and writes a JSON results summary.

Minimizing a deceptive, multimodal landscape

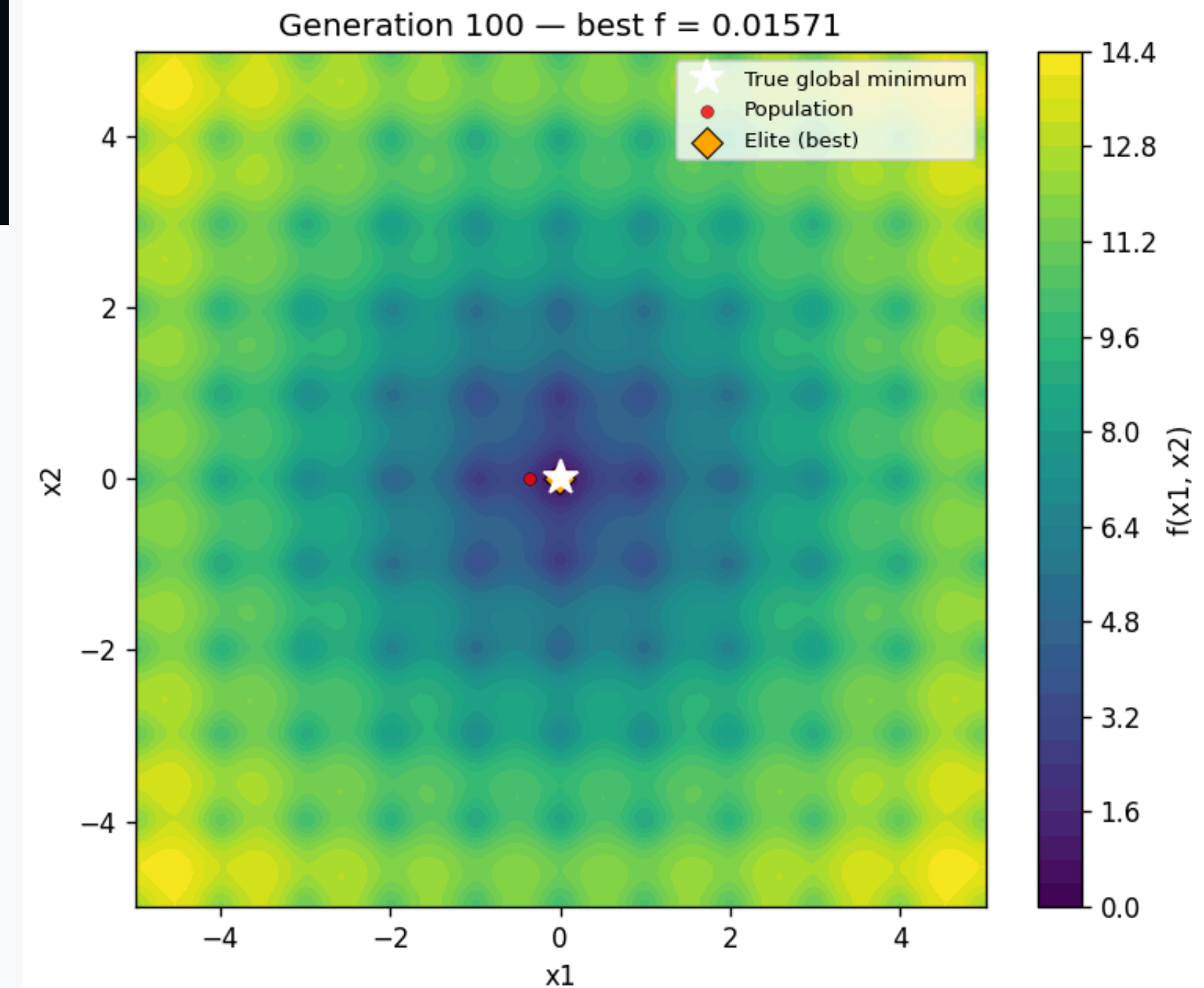
Mathematical Formula for 2D ($d = 2$)

$$f(x, y) = -a \exp\left(-b \sqrt{\frac{1}{2} (x^2 + y^2)}\right) - \exp\left(\frac{1}{2} (\cos(cx) + \cos(cy))\right) + a + \exp(1)$$

- Domain: $-5 \leq x_1, x_2 \leq 5$, searched with no derivative information
- Known global minimum: $f(0, 0) = 0$ — the target we're trying to reach
- The Ackley function itself is used directly as the fitness/objective value

Why it's hard

- ▶ **Flat outer plateau** — Slopes barely change far from the center — little guidance toward (0,0).
- ▶ **Dense local ripples** — Cosine terms create many recurring local minima that trap local-search methods.
- ▶ **Narrow central funnel** — A steep, tight pit near the origin needs fine-grained tuning to reach zero.



The Ackley surface: a bullseye of ripples around a deep central well at (0,0).

What goes into the system

Objective function

$$f(x_1, x_2) = -a \cdot \exp(-b\sqrt{0.5(x_1^2 + x_2^2)}) - \exp(0.5(\cos(cx_1) + \cos(cx_2))) + a + e$$

Param	Value	Meaning
a	20.0	Scaling constant
b	0.2	Exponential decay
c	2π	Cosine frequency
Domain	$[-5, 5]$	For both x_1 and x_2

Fitness = Ackley value itself • lower is better • target = 0 at (0,0)

Chromosome: $[x_1, x_2] \in \mathbb{R}^2$, each gene initialized uniformly at random inside $[-5, 5]$.

GA hyperparameters

50

Population size

100

Max generations

0.80

Crossover prob. (Pc)

0.05

Mutation prob. (Pm)

1 kept

Elitism

0.5

Mutation σ (Gaussian)

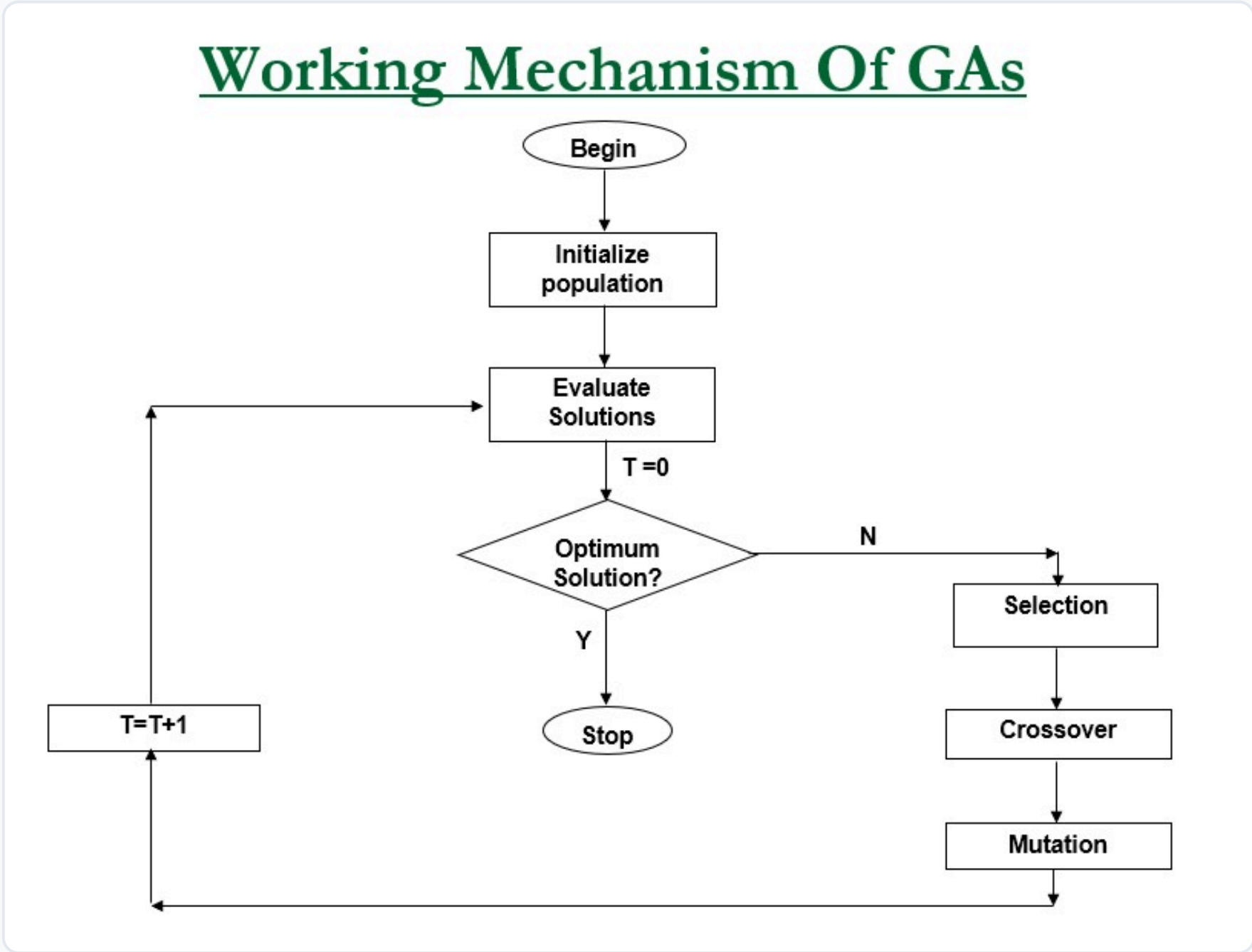
1e-6

Early-stop tolerance

42
(reproducibile)

Random seed

The standard GA flowchart



Reading the loop

Initialize population

`_init_population()` draws 50 chromosomes `[x1,x2]` uniformly from `[-5,5]`.

Evaluate solutions

`ackley_vec()` scores the whole population in one vectorized call.

Optimum solution? (Y/N)

Checks `best_fitness ≤ 1e-6` (Y → stop) or `generations = 100` (Y → stop).

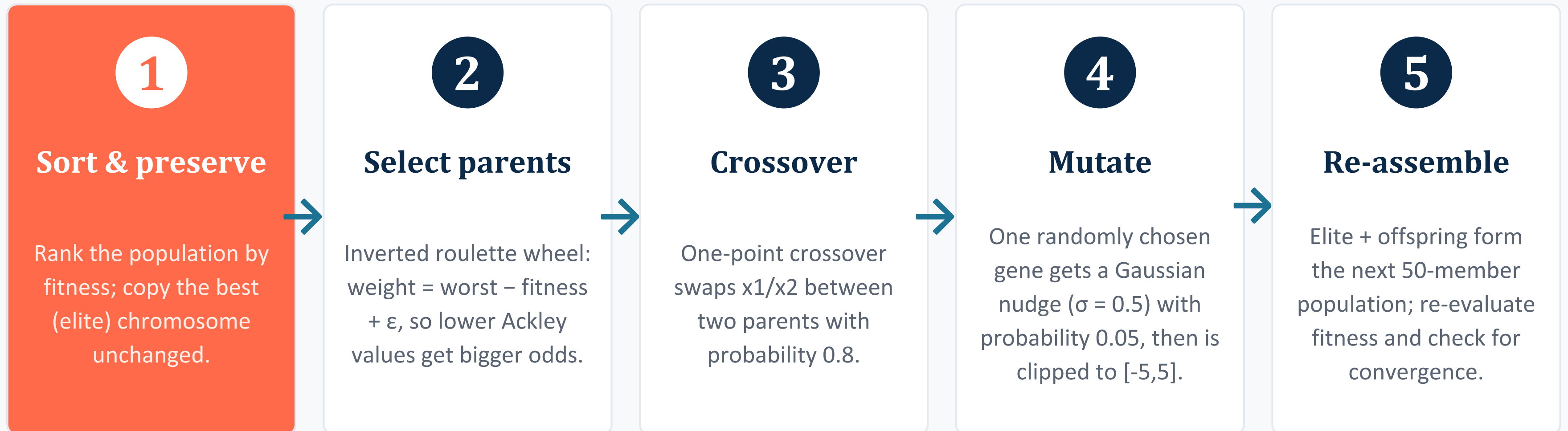
Selection → Crossover → Mutation

Inverted roulette wheel, one-point crossover ($P_c=0.8$), Gaussian mutation ($P_m=0.05$).

T = T + 1, loop back

Elite carried forward, new population re-evaluated, and the cycle repeats.

One generation of the algorithm, step by step



Loop repeats for up to 100 generations, or stops early once best fitness $\leq 1e-6$.

What actually happens inside each operator

1 · Selection

Inverted roulette wheel

$$w_i = (f_{\text{worst}} - f_i) + 1e-8$$

$$p_i = w_i / \sum w_k$$

Worked example (3 chromosomes):

A: $f=0.4 \rightarrow w=2.6 \rightarrow p=0.65$

B: $f=1.5 \rightarrow w=1.5 \rightarrow p=0.375$

C: $f=3.0$ (worst) $\rightarrow w=0.0 \rightarrow p \approx 0$

A is closest to zero, so it dominates the wheel; C (the worst) gets almost no chance. Parents are drawn with replacement from this distribution.

2 · Crossover

One-point, applied with $P_c = 0.8$

A 2-gene chromosome has exactly one interior cut point — between x_1 and x_2 — so the children simply trade genes:

$$P1 = [2.10, -3.40]$$

$$P2 = [-1.05, 0.90]$$

$$\rightarrow C1 = [2.10, 0.90]$$

$$\rightarrow C2 = [-1.05, -3.40]$$

Roll $< 0.8 \rightarrow$ crossover fires and children replace the parents; roll $\geq 0.8 \rightarrow$ parents pass through unchanged (copied as-is).

This mixes x_1 from one parent with x_2 from another — the only recombination possible with a 2-gene, real-valued chromosome.

3 · Mutation

One-gene Gaussian, applied with $P_m = 0.05$

- 1 Copy the chromosome (never mutate the parent array in place).
- 2 Pick one of the 2 gene indexes at random.
- 3 Draw $\Delta \sim N(0, \sigma^2)$ with $\sigma = 0.5$.
- 4 Add Δ to the chosen gene only.
- 5 Clip the result back into $[-5, 5]$ with `np.clip`.

$$\text{gene } x_2 = 4.85$$

$$\Delta = +0.9 \rightarrow 5.75 \rightarrow \text{clip} \rightarrow 5.0$$

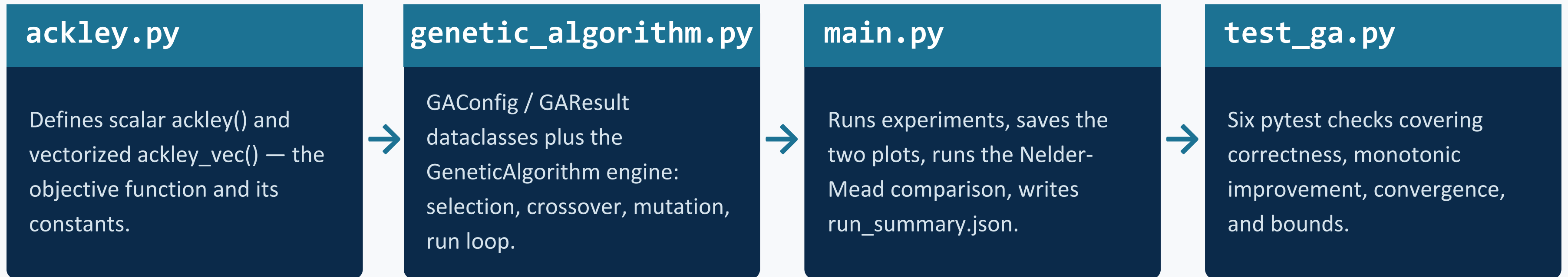
Small, localized nudges maintain diversity and let the population fine-tune around a promising basin without wild jumps.

Inside run(): the full generation loop, in order

- 1 `np.argsort(fitness)` ranks the 50×2 population array ascending.
- 2 `population[:elitism]` is copied verbatim into `next_population`.
- 3 Repeat: draw a parent pair via roulette → crossover (P_c) → mutate each child (P_m).
- 4 Keep appending children until `next_population` reaches size 50.
- 5 `next_population = elite + offspring`, replacing the current population.
- 6 `ackley_vec(next_population)` re-evaluates all 50 fitness values at once.
- 7 `best_fitness` / `best_chromosome` update only if the new best is lower.
- 8 `history_best` and `history_mean` each get one new value appended.
- 9 If verbose, print progress at generation 1 and every 10th generation.
- 10 Stop if `best_fitness` $\leq$ `convergence_tol` ($1e-6$) or generations = 100.

Elitism means step 7's best-so-far value is mathematically non-increasing generation over generation — the recorded curve can only flatten or drop.

Four Python modules, four libraries



Libraries

NumPy	SciPy	OpenPyXL	PyTest
Pandas	Matplotlib	ImageIO	

Run it: `cd src → pip install -r requirements.txt → python main.py` | Test it: `python -m pytest test_ga.py -v`

OUTPUT

What the program produced (seed 42)

[-0.00185, 0.00496]

Best chromosome

0.015713

Best fitness

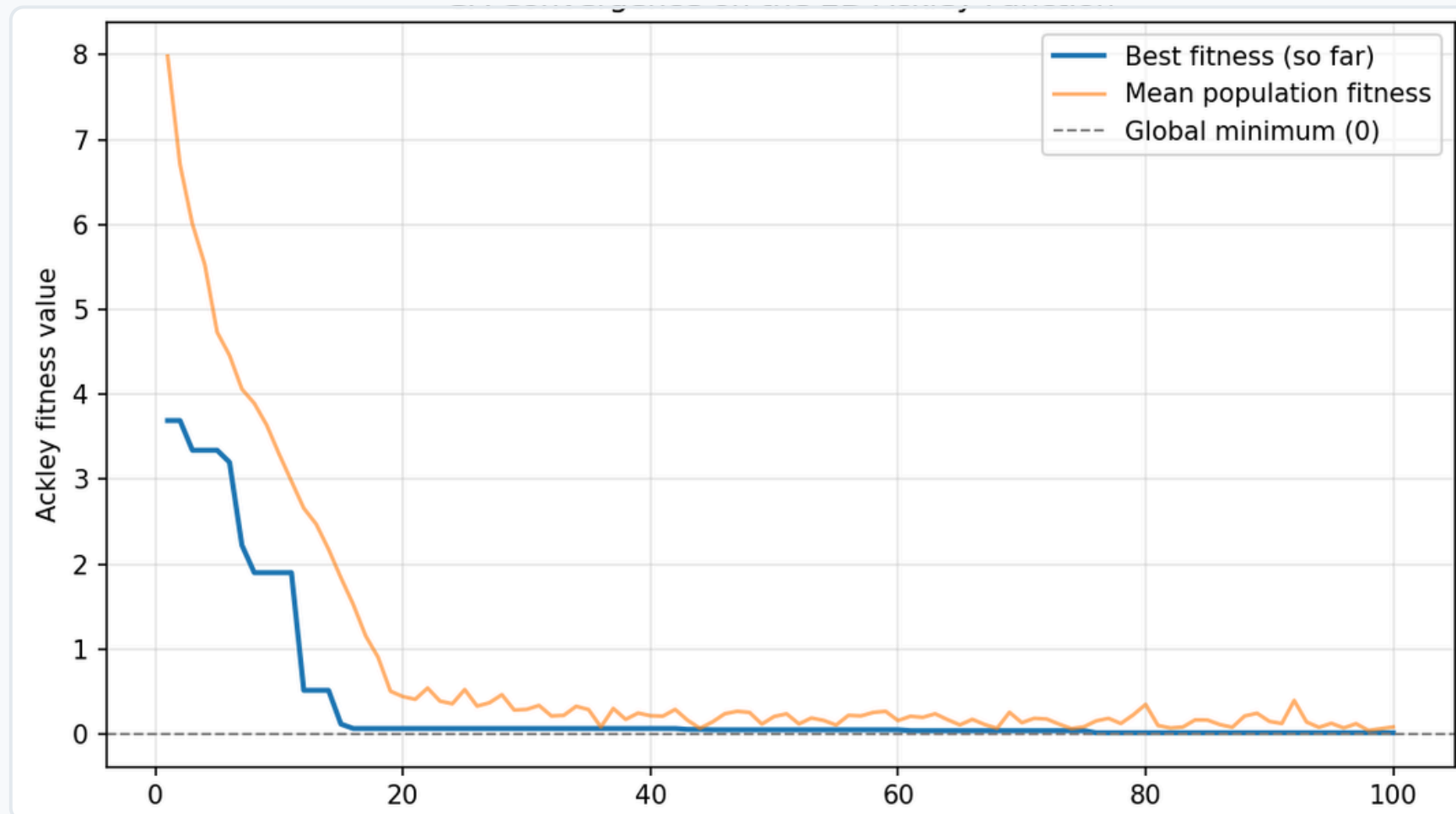
100

Generations run

No

Converged ($\leq 1e-6$)

Convergence over 100 generations



Also produced by the run

convergence_curve.png

Best-so-far & mean fitness vs. generation, with a zero reference line.

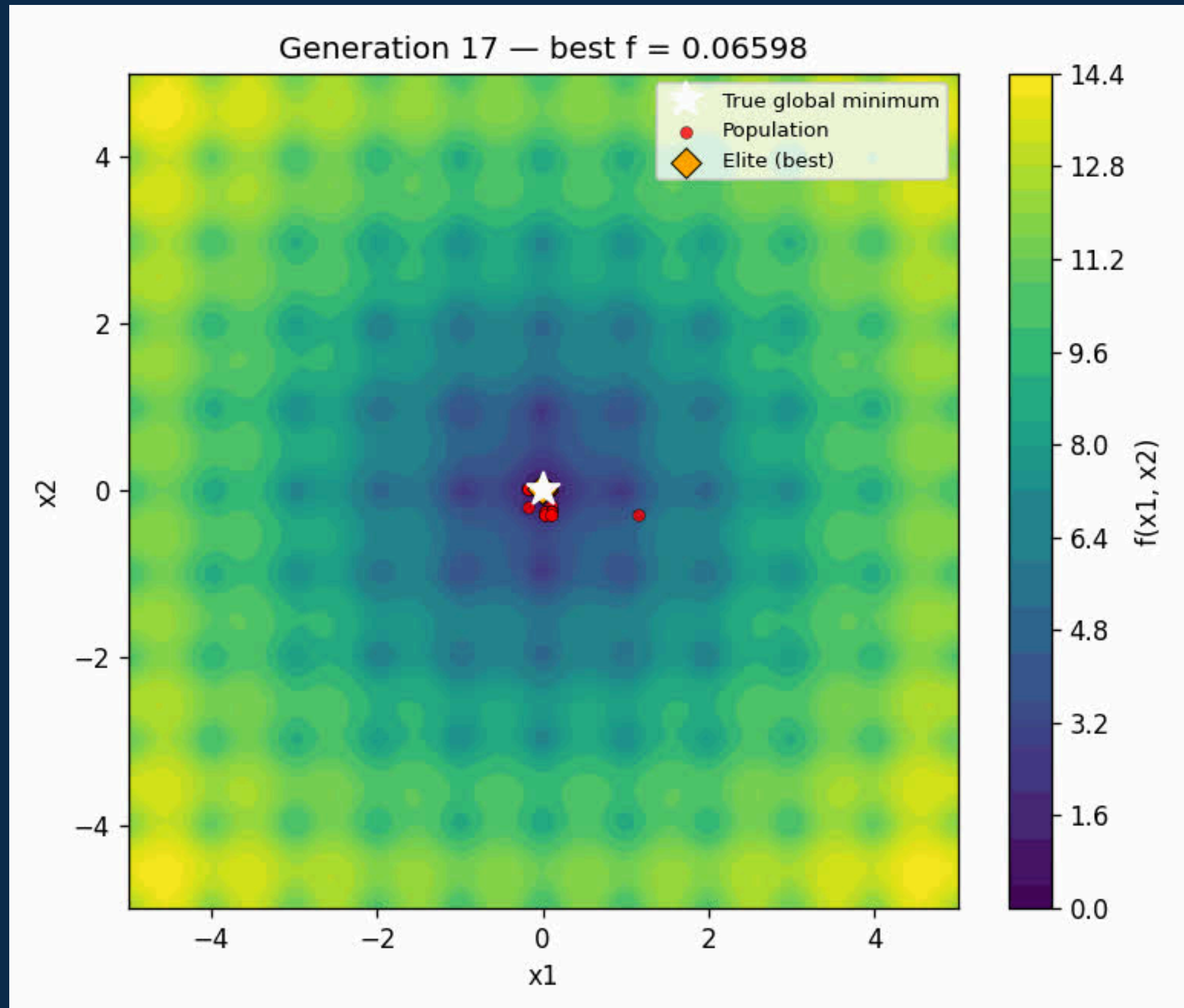
ackley_surface_solution.png

Filled contour of the landscape with the true minimum and GA solution marked.

run_summary.json

Machine-readable record of settings, GA result, classical comparison, and reduced-generation runs.

The GA solution on the Ackley surface



What the results tell us

GA vs. classical Nelder-Mead

Metric	Genetic Algorithm	Nelder-Mead
Start	Random population (N=50)	[4.0, -4.0]
Result	[-0.00185, 0.00496]	[3.983, -3.983]
Fitness	0.015713	10.998262
Outcome	Reached the global basin	Trapped in a local ripple

Why: Nelder-Mead only follows local slope from one starting point, so it settles in the nearest ripple. The GA keeps 50 points spread across the whole domain, so some always land near the true basin.

Automated verification

6 / 6 tests passed · 1.09s · Python 3.14.3

- ✓ $f(0,0) \approx 0$ within $1e-9$
- ✓ Fitness is positive away from the origin
- ✓ Scalar and vectorized evaluations agree
- ✓ Best-so-far fitness never increases
- ✓ 100-gen run reaches fitness below 1.0
- ✓ Final chromosome stays within $[-5, 5]$

Interpretation: the run is close to the mathematical optimum but not exact — typical for a stochastic search. A different seed or a longer budget could close the remaining gap.

The GA reached the basin the local optimizer couldn't

1

Optimization success

Navigated the multimodal Ackley landscape to $f \approx 0.0157$, right beside the true minimum.

2

Global beats local, here

Outperformed Nelder-Mead, which stayed trapped in a local ripple from its starting point.

3

Verified & reproducible

Six pytest checks pass; a fixed seed makes the run repeatable end to end.

Thank you for your attention!